

0:00 – 0:45 | Section 1: Introduction & High-Level Overview

[WHAT TO SHOW ON SCREEN]

- Start recording on your live application screen (<http://localhost:5174> or your **Vercel URL**).
- Show the **AIVOA Dashboard** with KPI cards, charts, and recent complaints.

"WHAT TO SAY" (Narration)

*"Hello everyone! Welcome to the demonstration of **AIVOA** — an AI-powered Quality Management System built specifically for pharmaceutical companies.*

In the pharma industry, customer complaints—like discolored capsules or damaged seals—come in as unstructured emails, PDFs, or manual reports. Processing these complaints manually is slow, error-prone, and risky for regulatory compliance.

AIVOA automates this entire pipeline using a multi-node AI Agent workflow powered by LangGraph, FastAPI, and React. Let's take a look at how it works from start to finish!"

0:45 – 1:45 | Section 2: Live UI Demonstration (User Input & AI Extraction)

[WHAT TO SHOW ON SCREEN]

1. Click on "**New Complaint**" or "**AI Copilot**" in the sidebar.
2. Show the split-screen layout: **AI Copilot Chat** on the left, **Complaint Form & Risk Assessment Card** on the right.
3. Paste this sample complaint into the chat:

"Apollo Pharmacy reported discolored capsules in Amoxicillin Capsules 500 mg. Batch number AMX240602. Manufacturing date March 2026. Expiry date February 2028. Please log this complaint."

4. Press **Enter / Send**.
5. Highlight how the form fields (Product Name, Batch #, Customer, Dates) automatically light up and fill in, while the **Risk Assessment Card** updates with severity and confidence scores.

"WHAT TO SAY" (Narration)

"Here on the New Complaint page, we have our AI Copilot on the left and the official GxP Complaint Form on the right."

*I'll paste a raw, plain-English complaint message from a pharmacy and hit send. Notice how within seconds, the AI Copilot analyzes the text, extracts key entities like Product Name, Batch Number, Customer, and Manufacturing Dates, and **automatically populates the form**.*

*At the same time, it calculates a **Completeness Score**, performs an automated **Risk & Severity Assessment**, and highlights the updated fields in green so the Quality Assurance officer can verify everything instantly."*



1:45 – 3:00 | Section 3: Frontend Code Walkthrough

[WHAT TO SHOW ON SCREEN]

1. Switch your screen to **VS Code**.
2. Open frontend/src/pages/CopilotPage.tsx or frontend/src/routes/AppShell.tsx.
3. Open frontend/src/features/complaints/complaintSlice.ts.
4. Open frontend/src/lib/axiosClient.ts.

"WHAT TO SAY" (Narration)

"Now, let me show you how the frontend code is structured.

*On the client side, we use **React with Redux Toolkit**.*

In CopilotPage.tsx, when a user sends a prompt or uploads a PDF document, the component calls our API client service.

In axiosClient.ts, we handle authentication headers and route requests to our backend endpoints. Once the AI responds, the payload is dispatched to our Redux slice complaintSlice.ts.

*We follow a strict **single-writer pattern**: only the AI response updates the draft state, ensuring state consistency across the UI while maintaining real-time field highlights and confidence indicators."*

3:00 – 4:45 | Section 4: Backend API & LangGraph Workflow

[WHAT TO SHOW ON SCREEN]

1. In VS Code, navigate to backend/app/api/copilot.py.
2. Open backend/app/langgraph/graph.py (or nodes folder backend/app/langgraph/nodes/).
3. Scroll through the nodes:
 - input_type_detection.py
 - entity_extraction.py
 - risk_assessment.py
 - duplicate_detection.py

"WHAT TO SAY" (Narration)

"Next, let's look at the backend heart of AIVOA.

Our backend is built using **FastAPI** in Python. When the request hits `/api/v1/copilot/chat` or `/copilot/upload`, it triggers a stateful **LangGraph multi-node workflow**.

Instead of relying on a single prompt, our AI agent processes complaints through 6 specialized nodes:

1. **Input Type Detection**: Determines whether the input is raw text, an email, or an uploaded PDF document.
2. **Entity Extraction**: Uses Groq/LLMs to extract GxP fields like batch numbers, dates, and dosage forms.
3. **Data Validation & Completeness**: Checks for missing mandatory fields according to regulatory guidelines.
4. **Risk & Severity Assessment**: Evaluates patient safety impact and assigns severity levels like Critical, Major, or Minor.
5. **Duplicate Detection**: Scans existing database records to catch recurring batch issues.
6. **CAPA Recommendations**: Generates suggested Corrective and Preventive Actions for the QA team.*

Each node updates a shared state object, ensuring deterministic, traceable AI decision-making."

4:45 – 5:45 | Section 5: Database Persistence & Dashboard Analytics

[WHAT TO SHOW ON SCREEN]

1. Switch back to the **Browser**.
2. Click "**Commit Complaint**" on the form.
3. Show the success notification / redirect to **Dashboard**.

4. Point out how the **KPI cards** (Total Complaints, Open Complaints, Critical Risks) update, and show the complaint appearing in the **Recent Complaints** table.
5. Quickly click on "**Complaints**" page to show full list.

"WHAT TO SAY" (Narration)

*"Once the Quality Officer reviews the AI-extracted draft, clicking **Commit Complaint** persists the structured complaint into our database using **SQLAlchemy and SQLite/PostgreSQL**.*

*An automated audit trail record is generated for 21 CFR Part 11 compliance. As soon as it's committed, the **Dashboard Analytics** refresh in real time, updating monthly trend charts, severity donut charts, and high-risk alerts for management oversight."*

5:45 – 6:30 | Section 6: Summary & Conclusion

[WHAT TO SHOW ON SCREEN]

1. Open the **GitHub Repository** (<https://github.com/rizzit17/pharmassist>) or project root showing README.md.
2. Briefly show your live **Vercel** and **Render** deployment URLs.

"WHAT TO SAY" (Narration)

"To summarize: AIVOA bridges the gap between unstructured customer feedback and strict pharmaceutical compliance.

By combining a modern React frontend, FastAPI backend, and LangGraph multi-node AI workflows, AIVOA reduces complaint intake time from hours to seconds while maintaining 100% regulatory traceability.

The entire codebase is open-source on GitHub and fully deployed live on Vercel and Render. Thank you so much for watching!"